

Procesadores Superescalares: emisión fuera de orden y control de dependencias

Arquitectura de computadores - Semana 8

Grado en *Ingeniería Informática*

Departamento de Informática. Universidad de Jaén



Objetivos

General

Comprender el funcionamiento de la ejecución fuera de orden en procesadores superescalares y analizar el papel del renombramiento de registros y del buffer de reorden en la eliminación de dependencias y la preservación de la consistencia de resultados.

Específicos

- Reconocer las **limitaciones del paralelismo** a nivel de instrucción y sus implicaciones en el diseño del procesador.
- Describir las **políticas de orden en los procesadores superescalares**: emisión, ejecución y retirada.
- Explicar **la lógica de la emisión fuera de orden** y su ventaja sobre la emisión en orden.
- Identificar los **riesgos de tipo WAR y WAW** y su efecto en la ejecución paralela.
- Comprender el **mecanismo de renombrado de registros**, tanto en hardware como en software, para eliminar dependencias falsas.
- Analizar el **papel del buffer de reorden** (ROB) en la consistencia y en la lectura de operandos.
- Relacionar el **tamaño de la ventana de instrucciones** con la capacidad de paralelismo explotado por el procesador.

Limitaciones fundamentales del paralelismo



Limitaciones fundamentales del paralelismo

- La aproximación superescalar introduce la capacidad de ejecutar múltiples instrucciones en paralelo.
- Sin embargo, no se puede realizar la ejecución en paralelo de instrucciones de forma indiscriminada, ya que existen ciertas limitaciones a las que la arquitectura tiene que enfrentarse.
- Las limitaciones más importantes son:
 - Dependencia de datos verdadera – Riesgo RAW
 - Dependencia relativas a los saltos condicionales
 - Conflictos con los recursos
 - Riesgo de tipo WAR (Antidependencia) y WAW (dependencia de salida)

Limitaciones fundamentales del paralelismo

Dependencia verdadera

- Se produce una dependencia verdadera de datos entre dos instrucciones cuando **una instrucción no se puede ejecutar hasta que finalice la ejecución de la otra**, ya que el operando destino de la primera es operando origen de la segunda.
- La **segunda instrucción necesita un dato** producido por la primera, por lo que debe esperar a que concluya la ejecución de la primera para que esté listo el dato que necesita.
- Ambas instrucciones no pueden ejecutarse en paralelo.
- En este caso se dice que existe una dependencia verdadera de datos entre ambas instrucciones (también llamada dependencia de flujo o dependencia de escritura/lectura).
 - En el tema anterior es lo que hemos denominado **riesgo de datos**.

Limitaciones fundamentales del paralelismo

Dependencias relativas a los saltos condicionales

- La presencia de un salto condicional puede afectar al funcionamiento del cauce.
- Dicho salto puede realizarse o no y las instrucciones que siguen al salto no pueden ejecutarse hasta que se conozca el camino que tomará el flujo del programa.
- Las consecuencias de estas dependencias en cauces superescalares son más graves, ya que por cada ciclo de retraso se pierde un mayor número de oportunidades de procesar instrucciones.

Limitaciones fundamentales del paralelismo

Dependencias relativas a los saltos condicionales

- Si existen técnicas de **predicción de saltos**
 - En el **peor de los casos** supone un fallo en la predicción del salto una vez conocido el camino de la bifurcación.
 - En este caso, se tienen varios ciclos de penalización para regular el cauce, que comenzará a llenarse, tras la ejecución de la bifurcación, captando las instrucciones del camino correcto.
 - En el **mejor de los casos**, ante un acierto en la predicción del salto, las instrucciones que siguen al salto pueden haber sido captadas y decodificadas, pero su ejecución sufrirá el retraso necesario hasta que se conozca el camino de la bifurcación.

Limitaciones fundamentales del paralelismo

Conflicto de recursos

- Un conflicto en un recurso se da cuando dos o más instrucciones requieren el mismo recurso, al mismo tiempo.
 - En el tema anterior lo hemos denominado **riesgo estructural**.
- Ejemplos de recursos son los puertos del banco de registros, unidades funcionales, cachés, etc.
 - Por ejemplo cuando ambas instrucciones necesitan la misma unidad funcional para ejecutarse y solamente hay una de este tipo, solo puede ejecutarse una instrucción mientras que la otra debe esperar para poder acceder al recurso y poder ejecutarse.
- Estas dependencias se pueden solventar aumentando recursos, mientras que las dependencias verdaderas (riesgos de datos) nunca se pueden solventar aumentando recursos.

Limitaciones fundamentales del paralelismo

Veremos más adelante un
cronograma ilustrativo

- **Riesgo WAR (Write-After-Read): Antidependencia**
 - Un riesgo WAR ocurre cuando una instrucción intenta escribir en un registro antes de que otra instrucción haya terminado de leer ese mismo registro.
 - Esto puede causar problemas porque la instrucción que está leyendo podría depender de un valor que cambiará después debido a la escritura.
 - En consecuencia, el valor utilizado podría ser incorrecto y llevar a resultados incorrectos.
- **Riesgo WAW (Write-After-Write): Dependencia de salida**
 - Un riesgo WAW ocurre cuando dos instrucciones intentan escribir en el mismo registro en un orden incorrecto, es decir, una instrucción intenta escribir en un registro después de que otra instrucción haya escrito en el mismo registro.
 - Esto puede causar problemas ya que la instrucción que escribe en segundo lugar podría sobrescribir el valor calculado por la primera instrucción, lo que resultaría en **valores** incorrectos.

Limitaciones fundamentales del paralelismo

Paralelismo a nivel de instrucción y a nivel de la máquina

- El **paralelismo a nivel de instrucción** de un programa depende de la frecuencia con que ocurren las dependencias verdaderas de datos y las dependencias relativas a los saltos condicionales que haya en el programa.
- Depende de:
 - Instrucciones de una secuencia que sean independientes
 - La ejecución de dicha instrucciones pueda solaparse o reordenarse
- El **paralelismo de la máquina** es una medida de la capacidad del procesador para sacar partido del paralelismo a nivel de instrucciones.
- Depende de:
 - Número de instrucciones que se pueden procesar (captar, decodificar, etc.) al mismo tiempo
 - Capacidad del cauces del procesador en paralelo

Limitaciones fundamentales del paralelismo

Paralelismo a nivel de programa y a nivel de la máquina

- Un programa que tenga un bajo nivel de paralelismo a nivel de instrucción (es decir, una gran cantidad de dependencias entre sus instrucciones) no podrá sacar el máximo partido de una máquina con un alto grado de paralelismo, quedando infrutilizada dicha máquina.

<code>lw x1, 0(x0)</code>	}	Paralelismo = 3
<code>addi x3, x8, 1</code>		
<code>add x4, x9, x5</code>		

<code>addi <u>x3</u>, x10, 1</code>	}	Paralelismo = 1
<code>add x4, <u>x3</u>, x9</code>		
<code>sw x0, 0(<u>x4</u>)</code>		

Políticas de instrucciones en orden



Políticas de instrucciones en orden

- Los procesadores superescalares pueden **emitir y ejecutar múltiples instrucciones en paralelo**.
- Para ello, implementan un enfoque más sofisticado al **gestionar tres políticas de orden** en el procesamiento de sus instrucciones.
- Estas políticas de orden permiten que las instrucciones se manejen de manera más **eficiente, maximizando el uso de las unidades funcionales y mejorando el rendimiento** general.
- El procesador va a intentar **localizar instrucciones independientes** más allá del punto de ejecución en curso, que puedan introducirse en el cauce para ser ejecutadas en paralelo.

Políticas de instrucciones en orden

- Respecto a esto, son importantes tres ordenaciones:
 1. El orden en que se emiten las instrucciones.
 2. El orden en que se **termina** la ejecución de las instrucciones.
 3. El orden en que las instrucciones actualizan los contenidos de registros y posiciones de memoria (finaliza)
- Una **instrucción** se considerará **ejecutada** cuando termina su ejecución en la unidad funcional.
- Una **instrucción** se habrá **completado** (escritura/retirada) cuando se hayan modificado los valores del banco de registros o en memoria si es un almacenamiento.
- El procesador puede alterar cualquiera de los órdenes con respecto al orden que tendrían secuencialmente.
 - La única restricción es que el resultado debe ser correcto.
- La captación y decodificación siempre es en orden

Emisión en orden

- **Emisión en orden y retirada (completada) en orden**
 - La emisión de instrucciones se realiza en el orden en que se producen en el flujo del programa.
 - Aunque este método permite la captura de más de una instrucción por ciclo, su eficiencia es limitada, lo que reduce su aplicabilidad en la práctica.
 - Además, las instrucciones pueden verse interrumpidas debido a los riesgos y dependencias que surgen durante su procesamiento.

Emisión en orden

- Ejemplo 1 de emisión en orden y **retirada** en orden
 - **I1** requiere 2 ciclos para completarse, lo que puede impactar en el flujo general de ejecución al prolongar su tiempo de ocupación de recursos.
 - **I3** y **I4** compiten por la misma unidad de ejecución, lo que puede llevar a conflictos y retrasos en la ejecución de estas instrucciones si ambas intentan utilizar el mismo recurso simultáneamente.
 - **I5** depende del valor producido por **I4**, lo que introduce una dependencia que puede causar una interrupción en la emisión de **I5** hasta que **I4** haya completado su ejecución y el valor esté disponible.
 - **I5** y **I6** también compiten por la misma unidad de ejecución, lo que puede resultar en una contención adicional que afecte la eficiencia del procesador.
 - El sistema cuenta con **dos unidades de captación y escritura (retirada)**, así como **tres unidades de ejecución**, lo que permite un cierto grado de paralelismo.
 - En la decodificación no pueden entrar nuevas instrucciones hasta que las anteriores no hayan pasado a ejecutarse

Emisión en orden

- Ejemplo de emisión en orden y **retirada** en orden

Decodificar		Ejecutar			Retirada (WB)		Ciclos
I1	I2						1
I3	I4	I1	I2				2
I3	I4	I1					3
	I4			I3	I1	I2	4
I5	I6			I4			5
	I6		I5		I3	I4	6
			I6				7
					I5	I6	8
							9

Emisión en orden

- Ejemplo de emisión en orden y retirada fuera de orden

Decodificar		Ejecutar			Retirada (WB)		Ciclos
I1	I2						1
I3	I4	I1	I2				2
	I4	I1		I3	I2		3
I5	I6			I4	I1	I3	4
	I6		I5		I4		5
			I6		I5		6
					I6		7
							8
							9

Políticas de instrucciones en orden

- Especificamos las instrucciones:
 - $\underline{X3} \leftarrow X2 + X5$ (I1)
 - $X4 \leftarrow \underline{X3} + 1$ (I2)
 - $X3 \leftarrow X5 + 1$ (I3)
 - $X6 \leftarrow X3 + 1$ (I4)
- **I2 depende del resultado de I1 – dependencia real (RAW)**
 - **I2** depende del resultado de **I1**, ya que utiliza el valor de X3 que se calcula en **I1**. Esto significa que **I2** no puede ejecutarse hasta que **I1** haya finalizado su ejecución y X3 tenga el valor correcto.

Políticas de instrucciones en orden

- Especificamos las instrucciones:
 - $\underline{X3} \leftarrow X2 + X5$ (I1)
 - $X4 \leftarrow X3 + 1$ (I2)
 - $\underline{X3} \leftarrow X5 + 1$ (I3)
 - $X6 \leftarrow X3 + 1$ (I4)
- Si I3 completa antes que I1, la entrada de I4 será incorrecta, ya que la salida es dependiente (riesgo tipo WAW)
 - La instrucción **I3** también modifica X3, sobrescribiendo el valor calculado en **I1**.
 - Si **I3** se completa antes de que **I1** termine, entonces **I4** (que utiliza X3) obtendrá un valor incorrecto, lo que resulta en una **dependencia en el registro destino** (WAW). En este caso, ambas instrucciones (**I1** y **I3**) intentan escribir en X3, lo que puede causar problemas si no se gestionan adecuadamente.

Políticas de instrucciones en orden

- Especificamos las instrucciones:
 - $\underline{X3} \leftarrow X2 + X5$ (I1)
 - $X4 \leftarrow X3 + 1$ (I2)
 - $\underline{X3} \leftarrow X5 + 1$ (I3)
 - $X6 \leftarrow \textcolor{red}{X3} + 1$ (I4)
- Si I3 completa antes que I1, la entrada de I4 será incorrecta, ya que la salida es dependiente (riesgo tipo WAW)
 - **Consecuencias para I4 (WAR):**

Si **I3** se completa antes que **I1**, el valor que **I4** utilizará para calcular X6 será erróneo, ya que dependerá de un valor X3 que no corresponde a la ejecución correcta del programa.
 - Esta situación puede generar resultados inesperados y errores en el procesamiento.

Introducción a emisión fuera de orden y renombrado de registros



Políticas de instrucciones fuera de orden

- **Emisión fuera de orden y retirada fuera de orden**
 - Se basa en desacoplar el flujo de decodificación y el de ejecución.
 - Se puede seguir captando y decodificando instrucciones hasta que el cauce esté lleno.
 - Cuando una unidad de ejecución está disponible, se puede ejecutar una instrucción
 - Las instrucciones decodificadas esperan a ser emitidas/ejecutadas en una estructura denominada **Ventana de Instrucción**

Políticas de instrucciones fuera de orden

- Ejemplo de emisión fuera de orden y retirada fuera de orden

Decodificar		Ventana Instrucción	Ejecutar			Retirada (WB)		Ciclos
I1	I2							
I3	I4	I1, I2	I1	I2				1
I5	I6	I3,I4	I1		I3	I2		2
		I4,I5,I6		I6	I4	I1	I3	3
		I5		I5		I4	I6	4
						I5		5
								6
								7
								8
								9

Políticas de emisión de instrucciones

Dependencia de lectura-escritura (**Antidependencia**): I2-I3(X3)

- $X3 \leftarrow X3 + X5$ (I1)
 - $X4 \leftarrow X3 + 1$ (I2)
 - $X3 \leftarrow X5 + 1$ (I3)
 - $X7 \leftarrow X3 + X4$ (I4)
-
- I3 no debe ejecutarse antes de que empiece I2 ya que I2 necesita un valor en X3 e I3 cambia X3.

Renombrado de registros

- En el procesamiento de **instrucciones en procesadores superescalables**, múltiples instrucciones pueden estar en diferentes etapas de ejecución al mismo tiempo y pueden originar **dependencias de salida (WAW) y las antidependencias (WAR)**, debido a que el contenido de los registros puede no reflejar el flujo correcto del programa.
- Cuando se presentan estas dependencias, **puede originarse un riesgo en el cauce, produciendo una detención.**
 - La detención se produce cuando una o más instrucciones no pueden continuar su ejecución debido a **conflictos en el acceso al contenido a los registros.**
- Este hecho provoca **decrementar la eficiencia del procesador**, ya que se pierde tiempo valioso en el que podría haberse ejecutado otra instrucción. Es crucial **gestionar las dependencias de manera efectiva** para **minimizar estas detenciones y maximizar el rendimiento** del sistema.
- Es importante considerar que lo importante son los registros lógicos (valor) y los registros físicos pueden ser renombrados en el hardware

Renombrado de registros

- Ejemplo de registros
 - $X3a \leftarrow X3 + X5$ (I1)
 - $X4 \leftarrow X3a + 1$ (I2)
 - $X3b \leftarrow X5 + 1$ (I3)
 - $X7 \leftarrow X3b + X4$ (I4)
- Sin etiqueta (a,b) se refiere al registro lógico
- Con etiqueta es registro hardware asignado
- Elimina el riesgo WAR de antidependencia I2-I3(X3) y el riesgo WAW de la dependencia de salida I1-I3, así como I1-I4, ambas con X3.
- Para ello, necesitamos registros adicionales, más adelante veremos cómo se gestiona.

Emisión fuera de orden Visión Global



Políticas de emisión de instrucciones

- La emisión de instrucciones **se detiene** temporalmente cuando:
 - Surge una dependencia verdadera de datos
 - Hay un conflicto por un recurso
 - Si no hubiera predicción de saltos, hay una dependencia relativa a los saltos condicionales
 - Normalmente se utilizan técnicas de predicción de saltos
 - El paralelismo de instrucción del programa excede el paralelismo permitido por la máquina

Políticas de emisión de instrucciones

Limitaciones de la emisión en orden

- La emisión en orden tiene una seria limitación, el procesador decodifica instrucciones hasta que aparece una dependencia o conflicto.
- A partir de dicho punto no se decodificarán más instrucciones hasta que se resuelva dicho conflicto.
- **Importante problema en el rendimiento**
 - En el contexto de emisión en orden, el procesador no puede seguir buscando aquellas instrucciones independientes que pudieran haber más allá de este punto y que podrían ser ejecutadas, sacando así más partido a la máquina y aumentando la eficiencia de la ejecución.
 - De este problema, surge la política de **emisión desordenada**.

Políticas de emisión de instrucciones

Emisión desordenada

- Para permitir la emisión desordenada es necesario desacoplar las etapas del cauce de decodificación y emisión/ejecución. Esto se hace mediante un buffer donde la instrucción espera a ser emitida.
 - Si el buffer es una ventana general se denomina **ventana de instrucciones**.
 - Si el buffer es una ventana asociada a una UF específica se denomina **estación de reserva**.
- Cuando un procesador termina la decodificación de una instrucción, **coloca la instrucción decodificada** en la ventana de instrucciones o en la estación de reserva específica.
- Mientras la ventana de instrucciones/estaciones de reserva no se llenen, las vías de captación y decodificación pueden **captar y decodificar libremente**, sin verse condicionadas por la etapa de ejecución.
- Cuando una unidad funcional queda disponible en la etapa de ejecución se puede **emitir una instrucción** desde la ventana de instrucciones o estación de reserva a dicha unidad funcional, siempre que tenga los operandos disponibles.

Políticas de emisión de instrucciones

Emisión desordenada

- Podrá emitirse cualquier instrucción que:
 - a) Tenga los operandos disponibles
 - b) Esté disponible la unidad funcional que codifica la instrucción,
 - En el caso de que dos o más instrucciones tengan a la misma vez la UF disponible y los operandos disponibles, se le **dará prioridad** a aquella o aquellas instrucciones que se han captado y decodificado antes.
- El **procesador tiene capacidad de anticipación**, examinando las siguientes instrucciones del código para identificar instrucciones independientes que puedan ser ejecutadas.
 - Esta capacidad **será mayor cuanto mayor sea el tamaño de la ventana de instrucciones** (al aumentar dicho tamaño es posible realizar la búsqueda de instrucciones independientes sobre un mayor número de instrucciones).
 - Se reduce la probabilidad de que el cauce tenga que pararse, y aumenta la eficiencia de la ejecución.
- Las instrucciones se emiten desde la ventana de instrucciones/estación de reserva sin tener en cuenta el orden de las instrucciones en el programa.
 - Una estación de reserva es una ventana de instrucción específica de una unidad funcional
 - La única restricción es que el programa funcione correctamente.

Políticas de emisión de instrucciones

- Para mitigar los riesgos WAR y WAW, los procesadores utilizan técnicas como el **renombrado de registros y la reorganización de instrucciones** para asegurarse de que las instrucciones se finalicen en el orden correcto y que los resultados sean coherentes y preciso. Riesgos WAR y WAW:
 - **Riesgo WAR (Write-After-Read):** Un riesgo WAR ocurre cuando una instrucción intenta escribir en un registro antes de que otra instrucción haya terminado de leer ese mismo registro. Esto puede causar problemas porque la instrucción que está leyendo podría depender de un valor que cambiará después debido a la escritura. En consecuencia, el valor utilizado podría ser incorrecto y llevar a resultados incorrectos.
 - **Riesgo WAW (Write-After-Write):** Un riesgo WAW ocurre cuando dos instrucciones intentan escribir en el mismo registro en un orden incorrecto, es decir, una instrucción intenta escribir en un registro después de que otra instrucción haya escrito en el mismo registro. Esto puede causar problemas ya que la instrucción que escribe en segundo lugar podría sobrescribir el valor calculado por la primera instrucción, lo que resultaría en valores incorrectos.

Políticas de emisión de instrucciones

Emisión desordenada

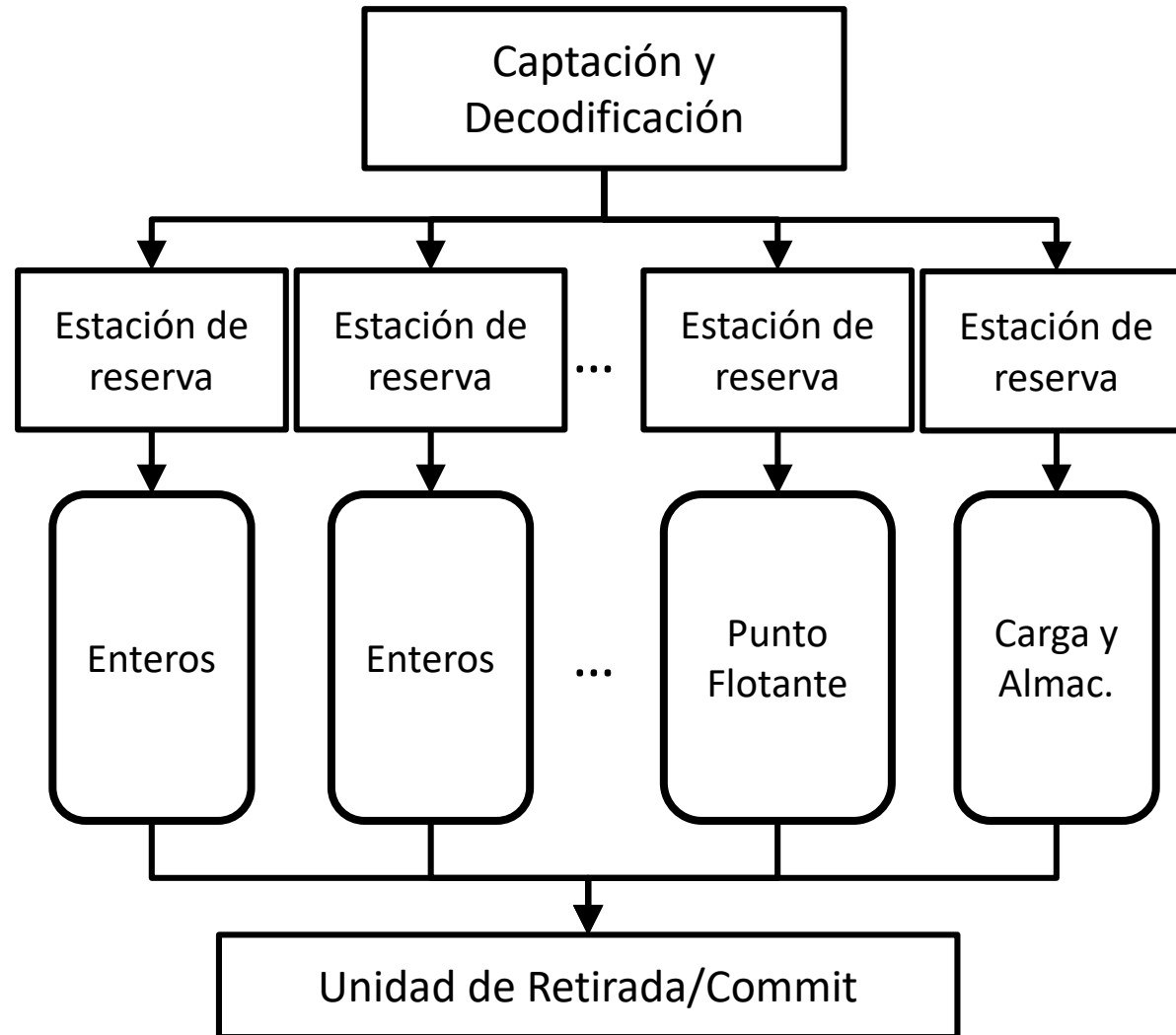
- El hecho de permitir la **emisión desordenada** de instrucciones y la retirada desordenada puede originar **riesgos WAR y riesgos WAW**.
- La naturaleza de estas dependencias es diferente a la de las dependencias de datos verdaderas, que reflejan el flujo de datos a través de un programa y su secuencia de ejecución.
- Las dependencias WAR y WAR, como sabemos, **no son verdaderas dependencias**, ya que surgen porque los valores de los registros no pueden reflejar ya la secuencia de valores establecida por el flujo del programa.
- Por tanto, para evitar este tipo de riesgos es necesario que las **instrucciones se completen** (modificado los valores del banco de registros o de memoria si es un almacenamiento) de **manera ordenada**.

Políticas de emisión de instrucciones

- Los procesadores superescalares, al permitir la emisión de instrucciones fuera de orden, suelen implementar una **consistencia de procesador fuerte y una consistencia de memoria débil**.
 - Esto se hace para aumentar el rendimiento y evitar problemas como los riesgos de escritura después de lectura (WAR) y escritura después de escritura (WAW).
- La **consistencia de procesador fuerte** asegura que las instrucciones se completen en el orden secuencial y se escriban en el banco de registros en el mismo orden en el que se captaron/decodificaron, lo que evita conflictos y problemas de dependencia entre instrucciones.
- La **consistencia de memoria débil** permite que las operaciones de lectura y escritura en memoria ocurran en un orden diferente al secuencial del flujo de instrucciones, lo que aumenta la eficiencia al ejecutarse de manera paralela
 - Se tiende a que las escrituras se realicen en orden y las lecturas se puedan adelantar, siempre que se garantice que se lee un dato correcto.

Procesador Superscalar – Ejemplo 1

Orden



Fuera de Orden

Orden

Orden de decodificación
(I1) `addi t0 t0 t1`
(I2) `lw t2 0(a0)`
(I3) `sub t4 a1 a0`
(I4) `addi t0 t0 t1`

Orden de emisión (fuera orden)
1. `sub` AND `addi` (1) AND `load`
2. `addi` (2)

Orden de commit (completadas / retiradas)
(I1) `addi t0 t0 t1`
(I2) `lw t2 0(a0)`
(I3) `sub t4 a1 a0`
(I4) `addi t0 t0 t1`

Procesador Superscalar – Ejemplo 2

- Permitir que la CPU emita/ejecute instrucciones fuera de orden para evitar riesgos
 - Pero siempre graba el resultado en los registros en orden (consistencia)
 - Puede iniciar sub mientras add está esperando lw
 - Especialmente si falla la caché, puede ejecutar muchas instrucciones

```
# Cargar el valor en la dirección s2 + 20 en $t0
lw t0, 20(s2)

# Sumar t0 y t2 y almacenar el resultado en t1
add t1, t0, t2

# Restar t3 de s4 y almacenar el resultado en s4
sub s4, s4, t3

# Establecer t5 a 1 si s4 es menor que 20; de lo contrario, establecerlo en 0
slti t5, s4, 20
```

Renombrado de registros

Visión Global



Renombrado de registros

Riesgos WAR y WAW

- Los riesgos WAR y WAW son realmente conflictos de almacenamiento.
- Son un tipo de **conflicto por los recursos** en el que varias instrucciones compiten por los mismos registros.
- La arquitectura debe resolver el conflicto deteniendo temporalmente alguna etapa del cauce.
 - La frecuencia de aparición de este tipo de instrucciones aumenta con el uso de las técnicas de optimización de registros, que intentan maximizar el uso de los registros.
- Un método para resolver tales conflictos se basa en la duplicación de recursos (registros).
 - En este caso, esta técnica se denomina **renombramiento de registros** y consiste en que el hardware del procesador asigne dinámicamente los registros (espacios para guardar los valores de los registros), que están asociados con los valores que necesitan las instrucciones en diversos instantes de tiempo.

Renombrado de registros

Técnica de renombrado de registros

- Cuando se ejecuta una instrucción, donde su operando destino es un registro, se le **asigna un nuevo registro físico** para almacenar el resultado, y las instrucciones posteriores que accedan a ese valor como operando fuente en ese registro, tienen que atravesar un proceso de renombramiento, donde **se revisan las referencias a registros**, para que definitivamente hagan referencia al registro físico que contiene el valor que se necesita.
- De este modo, diferentes instrucciones que tienen referencias a un único registro de la arquitectura (**registro lógico**), pueden referirse a diferentes registros reales (**registros físicos**), con valores diferentes.

Renombrado de registros

- Supongamos que tenemos dos instrucciones **sin renombrado de registros**:
`ADD X1, X2, X3` (Realiza la suma de X2 y X3 y almacena el resultado en X1).
`SUB X1, X4, X5` (Realiza la resta de X4 y X5 y almacena el resultado en X1).

X1 estaría bloqueado después de la primera instrucción hasta que se complete, lo que podría retrasar la segunda instrucción, la cual es independiente

- Con el **renombrado de registros**, se asignan registros físicos diferentes a X1 en cada instrucción, lo que permite que ambas instrucciones se ejecuten simultáneamente sin conflictos. Entonces, después del renombrado, las instrucciones se verían de la siguiente manera:
`ADD X1[rename1], X2, X3` (X1 se renombra como X1[rename1]).
`SUB X1[rename2], X4, X5` (X1 se renombra como X1[rename2]).

Ambas instrucciones utilizan registros renombrados, lo que permite su ejecución simultánea sin bloqueos. Posteriormente, los resultados finales se escribirían en el registro del banco de registros de manera ordenada.

Renombrado de registros

- Ejemplo de riesgos WAR (*Write After Read*):

(I1) **MUL** **x1**, x2, x3 ($X1 = X2 * X3$) – Termina en el ciclo 8

(I2) **SUB** x4, **x1**, x5 ($X4 = X1 - X5$) – En el ciclo 7 necesita el operador X1 de (1) y no de (3)

(I3) **ADD** **x1**, x6, x7 ($X1 = X6 + X7$) – Termina en el ciclo 6

- En un procesador fuera de orden, las instrucciones (I1) y (I3) pueden ejecutarse en paralelo.
- Sin embargo, con el riesgo WAR, puede ser problemático si I2 finaliza su ejecución después de que I3 haya completado la escritura del valor en X1. Así, se ejecuta la I3, después la I2 y, finalmente la I1
- Por tanto, este hecho podría resultar que I2 use un valor de X1 incorrecto, es decir, el valor de I3, en lugar del producido por I1
- Para evitar este riesgo se utiliza el renombrado de registros

Renombrado de registros

- Ejemplo de riesgo WAW (*Write After Write*):

(I1) **MUL** **x1**, **x2**, **x3** ($X1 = X2 * X3$) – Termina en el ciclo 8

(I2) **SUB** **x1**, **x4**, **x5** ($X1 = X4 - X5$) – Termina en el ciclo 6

- En este caso, la I2 intenta (**SUB**) escribir en X1 después de que I1 (**MUL**) iniciado su escritura en X1.
- Esto crea un riesgo WAW, ya que la I1 sobrescribir el resultado de I2.
- El renombrado de registros también se utiliza para resolver este tipo de riesgo en un procesador fuera de orden.

Renombrado de registros

Ejemplo ilustrativo

(1) **x3**=x3*x5

(2) x4=**x3**+1

(3) **x3**=x5+x6  Esta instrucción tiene una dependencia WAW con la instrucción (1) y una WAR con la instrucción (2)

(4) x7=x3*x4

Para calcular el nuevo valor de X3 no hace falta esperar a las instrucciones (1) y (2)

Versión modificada:

(1) **x3b**=x3a*x5

(2) x4a=**x3b**+1  Esta instrucción tiene una dependencia real con la instrucción (1)

(3) **x3c**=x5+r6

(4) x7a=**x3c***x4a  Esta instrucción tiene una dependencia real con la instrucción (3)

Renombrado de registros

- El renombrado de registros se puede implementar mediante:
 - Software (renombramiento estático): el compilador introduce variables auxiliares
 - Hardware (renombramiento dinámico): se utilizan bancos de registros y hardware de control que puede estar:
 - Banco independiente del banco de registros
 - Banco dentro del banco de registros
 - Incluido en un buffer de reorden, el cual se utiliza para retirar las instrucciones en orden.
- En los **buffer de renombrado (ROB)** se introducen unos bits de validez que indican si el contenido de un registro es válido o si va a ser sobrescrito.
 - Al captar los operandos, estos deben tener su **bit de validez** igual a 1 y el bit del registro de destino debe ponerse a 0.
- También se introducen los **bits con el valor de último**, que se refiere a la versión más reciente y actualizada de un registro que ha sido renombrado.
 - Esto es importante para asegurar que las instrucciones utilicen el valor correcto y actualizado de los registros, evitando resultados incorrectos debido a dependencias de datos.

Lecturas de operandos con banco de registros y ROB



Lecturas de operandos con banco de registros y ROB

- Se trabaja tanto con el banco de registros como con el buffer de renombrado (ROB).
- El ROB es una estructura de datos que permite guardar datos de varios registros que han sido renombrados
- Se puede adelantar la lectura fuera de orden de valores sin necesidad de escribir en el banco desordenadamente.
- Campos del ROB
 - **#**: Indica la línea
 - **Ev**: Indica si esa línea se está utilizando. Puede tener el valor 1, lo que indica que esa línea es válida, o el valor 0, lo que indica que la línea está anulada con información antigua.
 - **Dest**: Indica el número de registro que ha sido renombrado (P. Ej. 2 se refiere a X2)
 - **Valor**: Es el valor del registro indicado en **Dest**.
 - **Válido**: Toma el valor 0 o 1 para indicar si el contenido de valor es correcto (1) o incorrecto (0)
 - **Últ**: Toma el valor 0 o 1 para indicar si el registro destino ha sido renombrado por última vez (1) o no (0)

Banco de registros	
X1	10
X2	20
X3	30
X4	40
X5	50

Buffer de renombrado (ROB)					
#	Ev.	Dest.	Valor	Válido	Último
0	1	4	55	1	0
1	1	4	33	1	1
2	0				
3	0				
4	0				

Lecturas de operandos con banco de registros y ROB

- Las **celdas vacías** en el ROB representan valores nulos, lo que significa que esos espacios contienen bits que representan valores. Sin embargo, estos **valores son obsoletos**, ya que han sido invalidados debido a que el campo Ev. tiene un valor de 0.
- En ocasiones, es posible encontrar estos valores indicados, pero si la línea tiene un 0 en el campo Ev, se debe comprender que son obsoletos y, por lo tanto, no son válidos.

Banco de registros	
X1	10
X2	20
X3	30
X4	40
X5	50

Buffer de renombrado (ROB)					
#	Ev.	Dest.	Valor	Válido	Último
0	1	4	55	1	0
1	1	4	33	1	1
2	0				
3	0				
4	0				

Lectura de registros – Ejemplo ilustrativo 1

- **Ejemplo X2: Se requiere el registro X2.**
- En primer lugar, se busca en el ROB .
 - X2 no está renombrado porque las líneas que están marcadas con 1 en el campo Ev no tienen el valor 2 en el campo Dest.
 - Dado que el registro X2 no está renombrado, se procede a leer en el banco de registros
 - Se accede al banco de registros y se lee el valor, que es 20.

Banco de registros	
X1	10
X2	20
X3	30
X4	40
X5	50

Buffer de renombrado (ROB)					
#	Ev.	Dest.	Valor	Válido	Último
0	1	4	55	1	0
1	1	4	33	1	1
2	0				
3	0				
4	0				

Lectura de registros – Ejemplo ilustrativo 2

- **Ejemplo X4: Se requiere el registro X4.**
- En primer lugar, se busca en el ROB
 - Está renombrado porque las líneas 0 y 1 tienen marcado a 1 el campo Ev y tienen el valor 4 en el campo Dest (está renombrado X4)
 - Se lee aquella línea que su valor de Último está marcado a 1 (línea 1)
- Cuando se localiza la línea con el bit de Último a 1 (solamente habrá una o ninguna), hay dos opciones:
 - Si el bit de validez está a 1, se lee el contenido de Valor porque es correcto.
 - Si el bit de validez está a 0, se espera a que esté a 1 para leer el dato de esa línea del ROB con el valor correcto.
- En este caso, se lee el valor 33 porque el valor de Válido está a 1 en esa línea.

Banco de registros	
X1	10
X2	20
X3	30
X4	40
X5	50

Buffer de renombrado (ROB)					
#	Ev.	Dest.	Valor	Válido	Último
0	1	4	55	1	0
1	1	4	33	1	1
2	0				
3	0				
4	0				

Lectura de registros – Ejemplo ilustrativo 3

- **Ejemplo X3: Se requiere el registro 3**

- Las líneas 2 y 4 del ROB, tienen su bit de Ev. a 0 por lo que están invalidadas y los valores que hay son antiguos.
- Se lee el dato del banco de registros, el valor de R3 es 30 porque no está renombrado y en el banco de registro su valor es 30.

Banco de registros	
X1	10
X2	20
X3	30
X4	40
X5	50

Buffer de renombrado (ROB)					
#	Ev.	Dest.	Valor	Válido	Último
0	1	4	55	1	0
1	1	4	33	1	1
2	0	3	22	1	0
3	0	2	12	1	0
4	0	3	56	0	1

Lectura de registros – Ejemplo ilustrativo 4

- **Ejemplo X3: Se requiere el registro X3**

- R3 está renombrado en la línea 2 y es último
- La línea 4 no es valida porque Ev. tiene el valor 0, por lo que esa línea está obsoleta.
- El valor de X3 no está disponible en la línea 2 porque el bit de Válido está a 0.
 - Se espera a que la línea 2 cambie el bit de valido a 1
- El valor de 22 es un valor antiguo al estar marcado su bit de Valido a 0
- El valor de X3 se calcula en una instrucción que está esperando para ser ejecutada o está ejecutándose

Banco de registros	
X1	10
X2	20
X3	30
X4	40
X5	50

Buffer de renombrado (ROB)					
#	Ev.	Dest.	Valor	Válido	Último
0	1	4	55	1	0
1	1	4	33	1	1
2	1	3	22	0	1
3	0	2	12	1	0
4	0	3	56	0	0

Renombrado de registro – Ejemplo ilustrativo 1

- **Ejemplo Renombrado 1**

- Supongamos el ROB de la izquierda donde X3 está renombrado dos veces en la línea 0 y 1
- Se emite la siguiente instrucción:
 - ADD X3, X2, X1
- El ROB se actualiza con las siguientes dos tareas para renombrar X3 (ROB derecha)
 1. En la línea 1, cambiar el bit de último a 0 (ya que hay un nuevo R3).
 2. Insertar la nueva línea para renombrar a R3 con el bit de último a 1 y el bit de validez a 0.

Buffer de renombrado (antes de emitir)					
#	Ev.	Dest.	Valor	Válido	Último
0	1	3	55	1	0
1	1	3	33	1	1
2	0	5	22	0	0
3	0	2	12	1	0
4	0	3	56	0	1

Buffer de renombrado (después de emitir)					
#	Ev.	Dest.	Valor	Válido	Último
0	1	3	55	1	0
1	1	3	33	1	0
2	1	3	22 	0	1
3	0	2	12	1	0
4	0	3	56	0	1

Renombrado de registro – Ejemplo ilustrativo 1

- **Ejemplo Renombrado 2**

- Supongamos el ROB de la izquierda donde X3 está renombrado una vez en la línea 1.
- Se emite la siguiente instrucción:
 - ADD X3, X3, X1
- En primer lugar (ROB derecha):
 1. Se lee X3 del ROB que tiene el bit de último y de Válido a 1 (valor 33)
 2. Se lee X1 del ROB que tiene el bit de último de Válido a 1 (valor 55)
 3. Se invalida el bit de último en la línea 1
 4. Se renombra X3 en la línea 2

Buffer de renombrado (antes de emitir)					
#	Ev.	Dest.	Valor	Válido	Último
0	1	1	55	1	1
1	1	3	33	1	1
2	0	5	22	0	0
3	0	2	12	1	0
4	0	3	56	0	1

Buffer de renombrado (después de emitir)					
#	Ev.	Dest.	Valor	Válido	Último
0	1	1	55	1	1
1	1	3	33	1	0
2	1	3	22	0	1
3	0	2	12	1	0
4	0	3	56	0	1

Recursos

Descripción

Para ampliar tus conocimientos sobre los contenidos de esta semana te recomendamos que consultes los recursos indicados a continuación.

Recursos

- Arquitectura de Computadores de Julio Ortega, Alberto Prieto y Mancia Anguita. Ediciones Paraninfo – 978849732274